

Exam: Feedback Vertex Set Problem

Abstract

This report presents an investigation into the Feedback Vertex Set (FVS) problem, a well-known NP-complete graph problem. The FVS problem aims to find the smallest set of vertices in a graph whose removal makes the graph acyclic. Three different approaches were implemented and analyzed for solving the FVS problem: an exact algorithm utilizing parameterized analysis, an approximation algorithm, and a heuristic. Additionally, parallelization techniques were applied to enhance the efficiency of the algorithms. The report provides a detailed description of each algorithm, discusses their complexities, and evaluates their performance through experimentation on various instances.

1 Introduction

The Feedback Vertex Set (FVS) problem is an NP-complete combinatorial optimization problem with numerous applications in various domains, including network design, circuit optimization, and bioinformatics. Given an undirected graph, the objective of the FVS problem is to identify the smallest subset of vertices whose removal renders the graph acyclic. Formally, the problem is defined as follows: given a graph $G = (V, E)$, find a set $S \subseteq V$ such that the graph induced by $V - S$ contains no cycles.

In this report, we explore three different approaches to solve the FVS problem: an exact algorithm, an approximation algorithm, and a heuristic. The exact algorithm leverages parameterized analysis techniques to find the exact solution to the FVS problem in exponential time. The approximation algorithm aims to provide a solution with a guaranteed approximation factor, while the heuristic offers a practical approach to solving instances of the problem, albeit without optimality guarantees. Two

heuristic approaches are provided, one aims to reduce the time taken by the exact algorithm, and another provides an intuitive approach to finding a sub-optimal solution to the FVS problem.

Furthermore, recognizing the computational challenges associated with solving NP-complete problems, particularly on large-scale instances, parallelization techniques are employed to enhance the efficiency of three of the algorithms. Parallelization is implemented at the core of the chosen algorithm to exploit the computational resources efficiently.

2 Preliminaries

2.1 Minimum Feedback Vertex Set

Given an undirected graph $G=(V, E)$, a Feedback Vertex Set (FVS) is a subset of vertices $S \subseteq V$ such that the removal of these vertices from the graph results in an acyclic graph. The minimum feedback vertex set is to optimize the feedback vertex set to be as small as possible.

2.2 Parameterized analysis

The parameterized analysis is a powerful technique in computational complexity theory that focuses on studying the behavior of algorithms concerning specific parameters of the input. By analyzing how the algorithm's runtime or solution quality varies as these parameters change, the parameterized analysis provides insights into the inherent difficulty of solving certain problems. This approach enables a more nuanced understanding of algorithmic performance, particularly for problems where traditional complexity measures may not capture all relevant factors. The basic structure of the parameterized analysis is $RunningTime(n) = O(poly(n) * f(k))$, where k is a parameter based on the input and turns out to be

small in practical applications Sabine [2023]. Here $f(k)$ can be exponential in k .

3 Algorithm & Implementation

3.1 Exact Algorithm

Since the Feedback vertex set is an NP-complete problem, we need to check for all possible cases. But we can reduce the size of the input graph by using the following result:

Let's define an operation called **Short-Cut(v)** on a vertex v in graph G . This operation is commonly used in the literature. If the degree of vertex v is 1, then we remove v along with its incident edge from G . If the degree of vertex v is 2 and its two neighbors x and y are not adjacent, then we remove v and add an edge between x and y . We denote this operation as **Short-Cut(v)**. We say that the operation **Short-Cut(v)** can be applied to a vertex v if either the degree of v is 1, or the degree of v is 2 and its two neighbors are non-adjacent Iyad Kanj [2004].

Lemma 1: Let G be an undirected graph and let v be a vertex in G to which the operation **Short-Cut(v)** is applicable. Let G' be the graph resulting from applying **Short-Cut(v)**. Then $f(G') = f(G)$. Where $f(G)$ is the size of the minimum feedback vertex set of graph G .

Using Lemma 1 we apply the **Clear(G)** method that applies the **Short-Cut(v)** repeatedly until no more reductions are possible, that is no more nodes with degree = 1 or nodes with degree = 2 and non-adjacent neighbors exists. We define the number of elements remaining in the graph as k . Note that k can be also defined as the number of nodes with $degree > 2$ and nodes with $degree = 2$ with adjacent neighbors. But since the **Short-Cut(v)** is applied multiple times it's likely that the value of k less than the value we get by the above definition.

The exact algorithm itself is simply checking all possible combinations of elements that need to be removed to make the graph acyclic. Note that the order is in such a way that in i th iteration all possible combinations of feedback vertex of size i are checked, in $(i + 1)$ th iteration all possible combina-

Algorithm 1 Clear algorithm

```

1: procedure CLEAR( $G$ )    ▷ Reduced graph of  $G$ 
2:    $G' \leftarrow G$ 
3:   while Reduction-Possible( $G'$ ) do
4:     for all  $v$  in  $G'$  do
5:       Short-Cut( $v$ )
6:     end for
7:   end while
8:   return  $G'$ 
9: end procedure

```

tions of feedback vertex of size i are checked, and so on. It is important as it helps to avoid checking more elements further once a feedback vertex set is found.

Run time analysis:

In the worst case, the algorithm will need to check all possible subsets of the reduced graph G with k node, which takes $O(2^k)$ time. Each instance removes and adds 1 node to the adjacency matrix which takes $O(n)$ time and cycle detection takes place on every step which takes $O(n + e)$ (e : number of edges) time which is equal to $O(n^2)$.

Therefore, total time = $O((2^k) * (n^2))$. Here we see it follows the general form of parameterized analysis.

3.1.1 Parallelization of the exact algorithm

Parallelization of the exact algorithm is done using making the exact algorithm iterative instead of recursive. It is done by using set bits of a number: *bit_number* to determine the elements to be added in the feedback vertex set. The *bit_number* starts from 0 and the next number is determined by the below algorithm for Geeks [2021]. The main idea remains the same 1st we check all possible feedback vertex sets of i th size then we move to the $(i + 1)$ th iteration. Now as we are using an iterative approach it is straightforward to parallelize it. Also, our maximum possible *bit_number* is $(2^k) - 1$ as it has $(k - 1)$ number of set bits, so it is guaranteed that the graph will become acyclic till only 1 node is left. For p cores, to parallelize the algorithm we split the range of numbers from 0 to $(2^k) - 1$ in p ranges and run the p ranges in different cores of the processor. Each core updates the minimum feedback vertex set, and we get the final

minimum feedback vertex set.

Run time analysis:

The time complexity for the sequential algorithm as discussed above = $O((2^k) * (n^2))$. Here we parallelize the creating new sets to be tested to be the minimum feedback vertex set, therefore the time complexity of making all sets = $O(2^k)/p$ for p processors. Hence, the total time complexity = $O(((2^k) * (n^2))/p)$.

Algorithm 2 Find Next Number with Same number of Set Bits

```

1: function FIND_NEXT_NUM_SAME_SETBITS( $x$ )
2:    $rightOne \leftarrow x \wedge \neg(signed(x))$ 
3:    $nextHigherOneBit \leftarrow x + rightOne$ 
4:    $rightOnesPattern \leftarrow x \oplus nextHigherOneBit$ 
5:    $rightOnesPattern \leftarrow$ 
      $rightOnesPattern / rightOne$ 
6:    $rightOnesPattern \leftarrow rightOnesPattern \gg$ 
      $2$ 
7:   return    $nextHigherOneBit \vee$ 
      $rightOnesPattern$ 
8: end function
```

3.2 Approximate Algorithm

The approximate algorithm is a simple greedy approach yet it has a performance ratio bounded by $2\log(d) + 1$ where $d = \max$ degree in the graph Ann Becker [1996]. The algorithm is to keep removing vertices of the highest degree and updating the degree until we achieve an acyclic graph. Also, here the reduction using **Clear(G)** method is possible. Note that few changes have been made to the algorithm as compared to the cited paper: 1) **Clear(G)** method is added 2) The condition of stopping the algorithm is not until the graph is empty but we check on every step if the obtained graph is acyclic. The greedy approximation algorithm is as follows:

Runtime analysis:

The time to iterate through all vertices to remove and add to the feedback vertex set (in the worst case) is linear = $O(n)$. Time taken on each step due to finding the highest degree vertex = $O(n^2)$ and due to cycle checking = $O(n^2)$. Hence, total time complexity = $O(n^3)$.

Algorithm 3 Approximation algorithm

```

1: procedure APPROXIMATION( $G$ )   ▷ Minimum
   feedback vertex set of  $G$ 
2:   Clear( $G$ )
3:    $feedbackSet = \phi$ 
4:   while  $G$  is acyclic do
5:      $v \leftarrow$  highest-degree-vertex( $G$ )
6:      $feedbackSet \leftarrow (feedbackSet + v)$ 
7:     remove-node( $G, v$ )
8:   end while
9:   return  $feedbackSet$ 
10: end procedure
```

3.2.1 Parallelization of approximation algorithm

In the parallelization of the approximation algorithm, the function used to find the highest degree vertex is parallelized. For p cores, the range of numbers remaining in the graph has been divided into p different and equal ranges. Each of the ranges is then processed by different processors and we get a maximum degree vertex in a parallelized fashion.

Runtime analysis:

The time to find the highest degree in parallelized fashion = $O((n^2)/p)$. Our current algorithm considers the condition of stopping the loop when no cycle is detected using cycle detection which takes $O(n^2)$ time so the overall time complexity is not affected by parallelization, the time taken = $((n^2) + ((n^2)/p)) * n = O(n^3)$. But if we do not use this condition (as mentioned in the original cited algorithm), we can still achieve an approximation bound of $2\log(d) + 1$ where $d = \max$ degree in the graph. In this case, cycle detection is not used so the time complexity becomes = $O(((n^2)/p) * n) = O((n^3)/p)$.

3.3 Heuristic

3.3.1 Heuristic to reduce time of exact algorithm: Sorting

This is a simple heuristic applied to reduce the runtime of the exact algorithm. The idea is that it is better to put vertices with higher degrees in the set

to check if it is a feedback vertex set as nodes with higher degrees have more probability to be part of cycles or more number of cycles as compared to the nodes with lower degrees. The heuristic is to search for elements with higher degrees first as compared to the random order in comparison to the current exact algorithm. **It can reduce the time taken by the exact algorithm by a factor of 2** as it can find the feedback vertex set in the m th round where m is the number of elements in the feedback vertex set.

Runtime-analysis:

The extra time taken to find the degree of all nodes and sort in descending order of the degree = $O(n^2) + O(n * \log(n)) = O(n^2)$.

Total time = $O(2^k) * (n^2) + O(n^2) = O(2^k) * (n^2)$.

Parallelization:

This approach can be parallelized using the approach as the exact algorithm that was parallelized using set bits to find the set to be checked for feedback vertex set. Therefore, the running time for the parallel algorithm with p cores = $O((2^k) * (n^2))/p$.

3.3.2 Approximation Heuristic

The idea is to create 1 empty graph and 1 empty feedback vertex set in the beginning. Then we add vertices to the empty graph in the order of their degree. If a cycle is created by adding the node in the graph we remove it from the graph and add it to the feedback vertex set.

Runtime-analysis:

The time taken to check n nodes = $O(n)$. For every iteration cycle detection takes $O(n^2)$ time.

Therefore total time complexity = $O(n^3)$.

3.4 Implementation

The implementation of all the 7 algorithms mentioned above involves translating the algorithmic concepts into functional code. The programming language of choice, along with any external libraries, plays a crucial role in the efficiency and correctness of the implementation. For this project, I implemented the algorithms in C++, as it is fast and gives

Algorithm 4 Approximation Heuristic algorithm

```

1: procedure APPROXIMATION( $G$ )    ▷ Minimum
   feedback vertex set of  $G$ 
2:   Clear( $G$ )
3:    $feedbackSet \leftarrow \phi$ 
4:    $G' \leftarrow \phi$ 
5:    $i \leftarrow 0$ 
6:    $n \leftarrow \text{number of nodes}(G)$ 
7:    $array \leftarrow [0 - n]$ 
8:   sort according to degree( $G$ , array)
9:   while  $i < n$  do
10:     $v \leftarrow array[i]$ 
11:    add node( $G'$ ,  $v$ )
12:    if cycle found( $G'$ ) then
13:      remove node( $G'$ ,  $v$ )
14:       $feedbackSet \leftarrow (feedbackSet + v)$ 
15:    end if
16:     $i \leftarrow (i + 1)$ 
17:  end while
18:  return  $feedbackSet$ 
19: end procedure

```

good control over memory usage. The code ensures the generation of synthetic datasets generated randomly, implementing the above-mentioned sorting algorithm, and logging of relevant metrics: time of execution and value of k for every graph. Also, the language used for analyzing, cleaning, and plotting data is Python due to the wide variety of libraries available for it.

3.5 Algorithm Engineering Concepts

There are primarily 3 algorithm engineering concepts used in the EM Mergesort algorithm:

- **Parameterized analysis:**

Parameterized analysis was used where k was the parameter. Here k is upper bounded by: the number of nodes with $degree > 2$ and nodes with $degree = 2$ with adjacent neighbors

- **Parallelization:**

Parallelization has been used to speed up the algorithms by using all the cores of the CPU.

4 Experimental Evaluation

In this section, the experimental setup is described and the results are presented.

4.1 Data and Hardware

4.1.1 Dataset Generation

The dataset used is taken from PACE 2016 – Track B: Feedback Vertex Setpac. Also, to test randomly generated test cases are used. Since the problem is NP-complete and exponential for 4 algorithms a limit of $k \leq 30$ is set, above that the algorithm was taking too long. All the test cases are present in the repository.

4.1.2 Hardware

To evaluate all the algorithms an AMD Ryzen 5 5500u processor is used that has 12 threads and a base clock speed of 2.1 GHz. An internal memory of size 24 GB is used. For this evaluation number of cores has been considered = 8.

4.2 Results

We have the results after running the algorithms on the given dataset. The plots of all the results have been added to the report. Also, here are total running time of the instances were all the algorithm ran without timeout (in milliseconds):

Table 1: **Performance Comparison on 42 instances**

Method	Time (ms)
Exact Algorithm	1,484,116.062
Time Heuristic	1,487,023.335
Exact Parallel	1,440,432.976
Time Heuristic parallel	1,474,801.477
Approximation	249.824
Approximation parallel	458.398

The approximation algorithm and approximation heuristic algorithm are faster and hence were able to run on 189 instances. Below are the performance evaluations for them.

Table 2: **Performance Comparison on 189 instances**

Method	Time (ms)
Approximation Algorithm	36,508.02
Approximation Parallel Algorithm	49,525.6
Approximation Heuristic	4,588,153.03

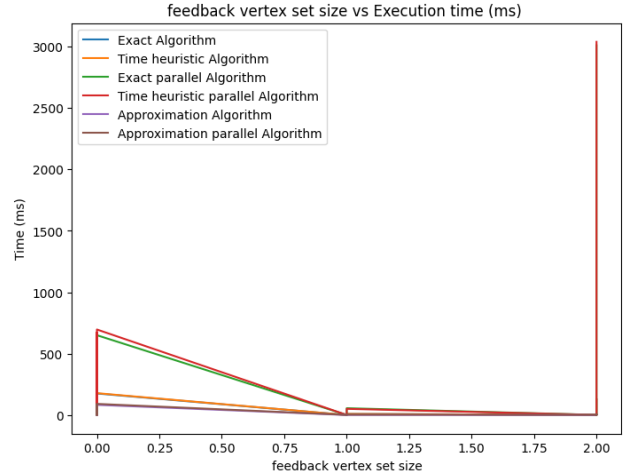


Figure 1: FVS Size vs time (15 points)

Also, the accuracy of the approximation algorithm = **13.54%**.

5 Discussion and Conclusion

We can see in the performance evaluation table that the Parallel algorithms are performing better than the conventional algorithms, although not p ($p = 8$) times, since parallelization is done by threading it might not use all the cores and depends on the CPU optimization.

In Figures 7 and 8 we can see there is a general trend that the time of execution is increasing with n but there are sudden drops when although we can see that k rises with n in in Figure 5 and Figure 6. This can be because few graphs even with a high value of k , can have a faster running time as removing very few nodes gives us the feedback vertex set.

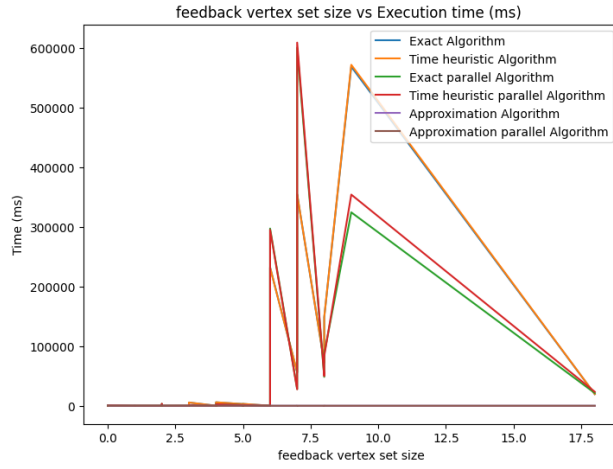


Figure 2: FVS Size vs time

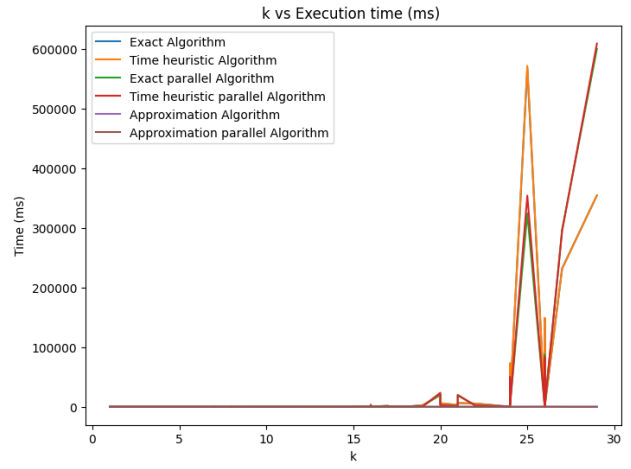


Figure 4: k vs time

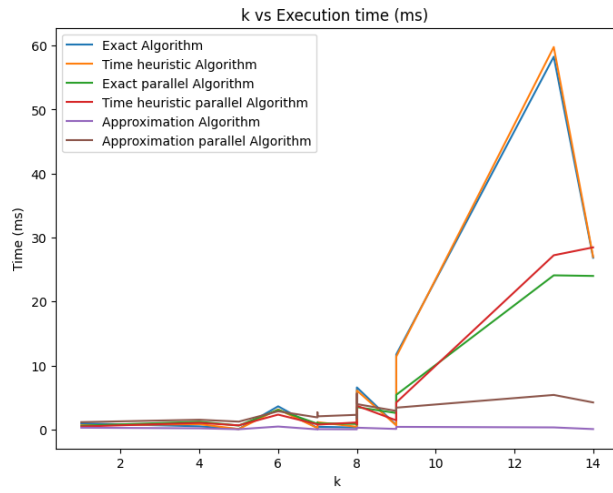


Figure 3: k vs time (15 points)

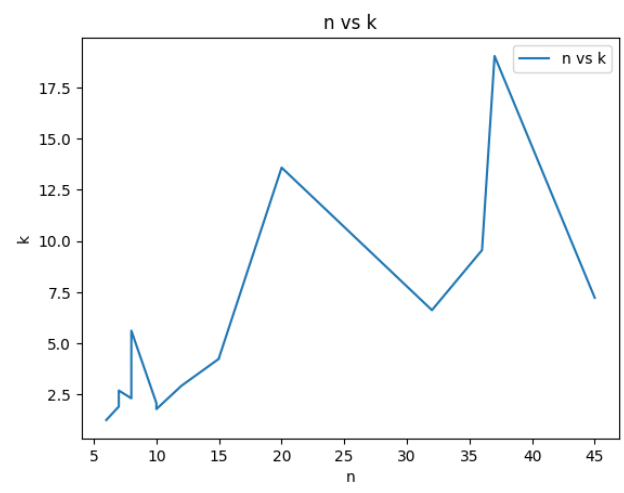


Figure 5: n vs k (15 points)

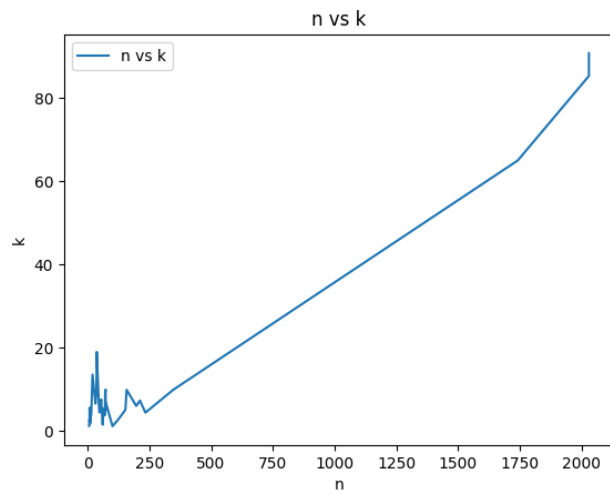


Figure 6: n vs k

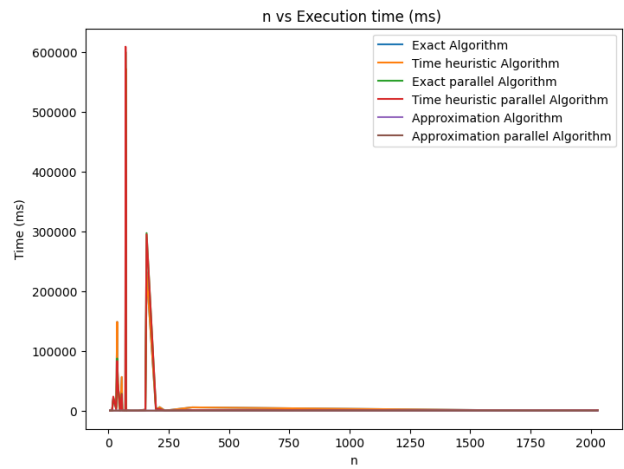


Figure 8: n vs time

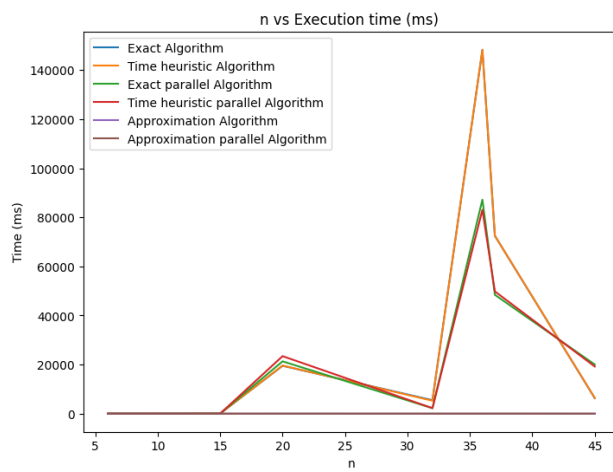


Figure 7: n vs time (15 points)

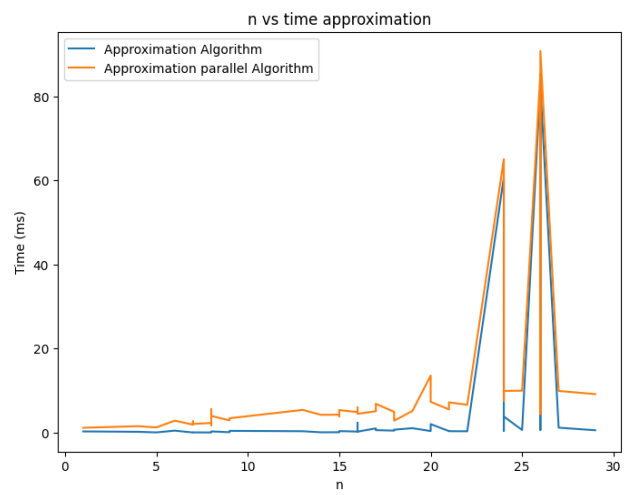


Figure 9: n vs time for approximation algorithms

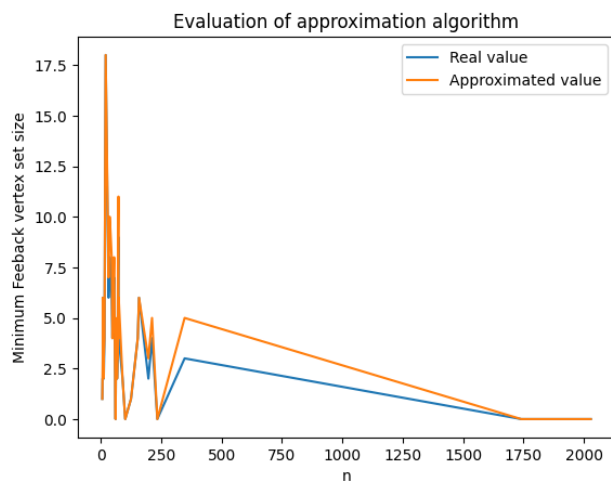


Figure 10: Evaluation of approximate algorithm

We see a similar trend when plotting time with respect to k in Figure 3 and Figure 4 but the drop is not as sharp as k is a better metric than n in this scenario. For the exponential algorithms, the algorithms are exponential in k and polynomial in n .

In Figure 1 and Figure 2 we can see that the feedback vertex set size is also not directly proportional to execution time, hence we can see that the execution time depends more on the structure of the graph for exponential algorithms.

For the approximation algorithm, we can see that the time steadily increases with the n except for 2 outliers which can be due to the high density of the graphs.

Figure 10 shows the accuracy of the approximation algorithm, the approximation curve matches the true values of the minimum size of the feedback vertex set, also we see that on average the approximation algorithm is giving us the size of the feedback cover with an error of about 13%. This number will increase as n increases as the approximation bound guarantees a limit of $2 * \log(d) + 1$, where d is the highest degree in the graph.

References

- Pace 2016 – track b: Feedback vertex set. URL <https://pacechallenge.org/2016/feedback-vertex-set/>.
- D. G. Ann Becker. Optimization of pearl’s method of conditioning and greedy-like approximation algorithms for the vertex feedback set problem, 1996.
- G. for Geeks. Next higher number with same number of set bits, 2021.
- M. S. Iyad Kanj, Michael Pelsmajer. Parameterized algorithms for feedback vertex set, 2004.
- S. Sabine. Lecture: 6. parameterized algorithms, 2023.